

Smart Lender – AI-Based Loan Approval Prediction System

Project Description:

Smart Lender is an AI-powered web application designed to assist banks and financial institutions in predicting loan approval decisions using Machine Learning. The system analyzes applicant information such as age, income, employment status, education, credit history, loan amount, and loan term to determine the likelihood of loan approval.

The application utilizes a Random Forest Classifier trained on historical loan application data to deliver accurate and reliable predictions in real time. Built using Flask, HTML, CSS, JavaScript, Bootstrap, and MySQL, the system provides a secure and user-friendly platform where users can register, log in, submit loan applications, and instantly receive prediction results.

The platform also maintains user authentication, stores prediction history in a MySQL database, and has been deployed on Render for public access. Performance testing using Apache JMeter confirms that the application can efficiently handle multiple concurrent requests with minimal response time. Smart Lender reduces manual effort in loan screening, improves consistency in decision-making, and assists financial institutions in identifying eligible applicants more efficiently.

Scenarios

Scenario 1: New Loan Applicant

A first-time user registers on the Smart Lender platform, logs in, enters their personal and financial details, and submits a loan application. The machine learning model evaluates the applicant's profile and predicts whether the loan is likely to be approved.

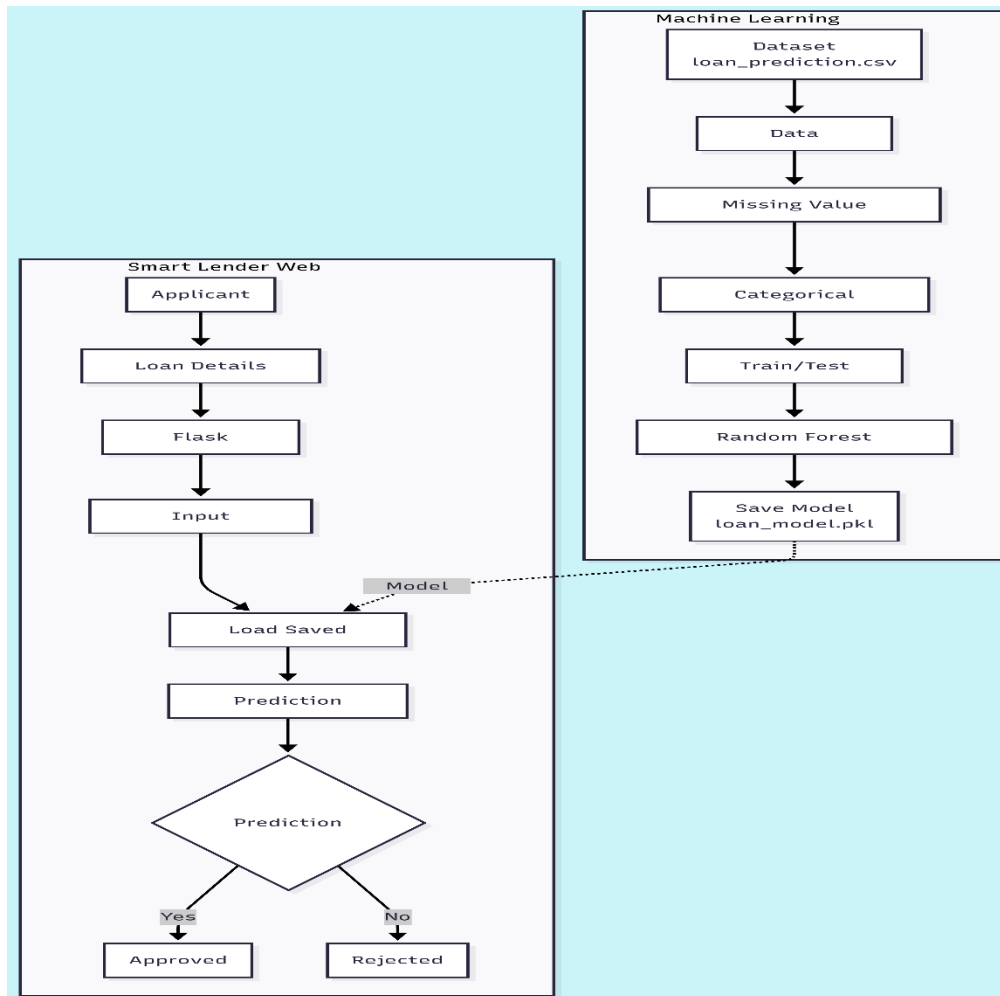
Scenario 2: Bank Officer Verification

A bank officer reviews an applicant's submitted details through the Smart Lender system. The AI model provides an instant prediction that assists the officer in making a faster and more consistent lending decision.

Scenario 3: Multiple Customer Requests

During peak banking hours, several customers submit loan applications simultaneously. Smart Lender processes multiple prediction requests efficiently, stores the results in the MySQL database, and delivers responses without significant performance degradation.

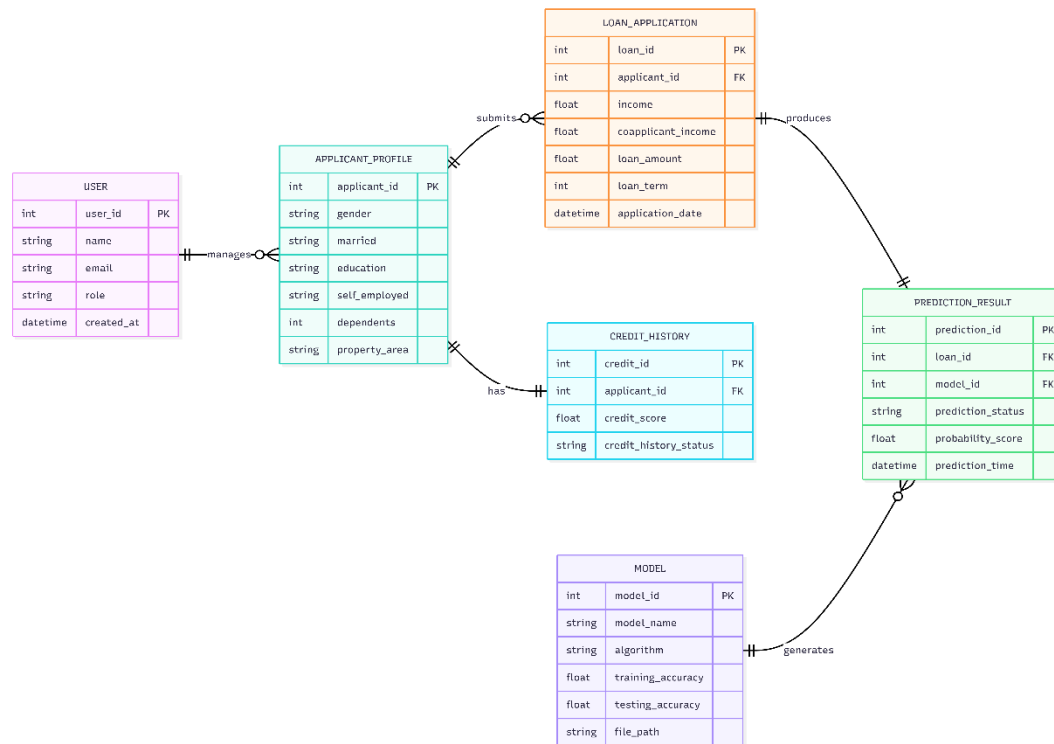
Technical Architecture:



Description: Smart Lender follows a three-tier architecture consisting of the Presentation Layer, Application Layer, and Data Layer. The system enables users to securely register, authenticate, submit loan details, and receive AI-based loan approval predictions. The Flask backend processes user requests, communicates with the Machine Learning model for prediction, and stores user information and prediction history in the database.

The architecture ensures modularity, scalability, and efficient communication between the user interface, business logic, machine learning engine, and database.

Entity Relationship (ER) Diagram:



Description

The Entity Relationship (ER) Diagram illustrates the logical structure of the Smart Lender database and the relationships among its core entities. The database is designed to securely manage user information and loan prediction records while ensuring efficient data retrieval and integrity.

The primary entity is the Users table, which stores user registration details such as User ID, Full Name, Username, Email Address, and Password. Each registered user is uniquely identified by a User ID, which serves as the primary key.

The second major entity is the Loan Prediction table, which stores loan application details including applicant information, financial attributes, prediction result, confidence score, and prediction timestamp. Each prediction record is uniquely identified using a Prediction ID.

Pre-requisites:

- **Flask Framework Knowledge:** Flask Documentation
- **Machine Learning with Scikit-learn:** Scikit-learn Documentation
- **Python Programming Proficiency:** Python Documentation
- **Pandas for Data Preprocessing:** Pandas Documentation
- **NumPy for Numerical Computing:** NumPy Documentation
- **Random Forest Classifier:** Scikit-learn Random Forest Documentation
- **Joblib for Model Serialization:** Joblib Documentation
- **HTML, CSS, and JavaScript Skills:** W3Schools HTML/CSS/JavaScript Tutorials

- **Bootstrap Framework:** Bootstrap Documentation
- **MySQL Database Knowledge:** MySQL Documentation
- **Git Version Control:** Git Documentation
- **GitHub Repository Management:** GitHub Documentation
- **Visual Studio Code Setup:** Visual Studio Code Documentation
- **Render Cloud Deployment:** Render Documentation
- **Apache JMeter for Performance Testing:** Apache JMeter Documentation

Project Workflow:

Milestone 1: Dataset Preparation and System Architecture

- **Activity 1.1:** Collect the loan prediction dataset from Kaggle and analyze its features.
- **Activity 1.2:** Perform data preprocessing by handling missing values, encoding categorical variables, feature scaling, and feature engineering.
- **Activity 1.3:** Design the overall architecture of the Smart Lender application, including the Flask backend, Machine Learning model, MySQL database, and web interface.
- **Activity 1.4:** Configure the development environment by installing Python, Flask, MySQL, required libraries, and setting up the project structure.

Milestone 2: Machine Learning Model Development

- **Activity 2.1:** Train and evaluate multiple Machine Learning algorithms such as Logistic Regression, Decision Tree, Random Forest, and XGBoost.
- **Activity 2.2:** Select the best-performing model based on accuracy and save the trained model using Joblib.

Milestone 3: Flask Application Development

- **Activity 3.1:** Develop the Flask backend, integrate the trained model, implement prediction routes, and connect the application with the MySQL database.

Milestone 4: Frontend Development

- **Activity 4.1:** Design responsive user interfaces for Login, Registration, Home, Loan Prediction, Dashboard, and About pages.
- **Activity 4.2:** Implement form validation, prediction result display, profile management, and navigation between pages.

Milestone 5: Deployment and Performance Testing

- **Activity 5.1:** Deploy the Smart Lender application on Render.
- **Activity 5.2:** Test the application using Apache JMeter and analyze performance using Summary Report, Aggregate Report, and Graph Results.

Milestone 6: Conclusion

Milestone 1: Dataset Preparation and System Architecture

In this milestone, we focus on preparing the loan prediction dataset and establishing the architecture of the Smart Lender application. This includes understanding the dataset, performing preprocessing, designing the system architecture, and configuring the complete development environment. These activities ensure that the machine learning model receives high-quality data and that the application is built on a scalable and modular foundation.

Activity 1.1: Collect the Loan Prediction Dataset

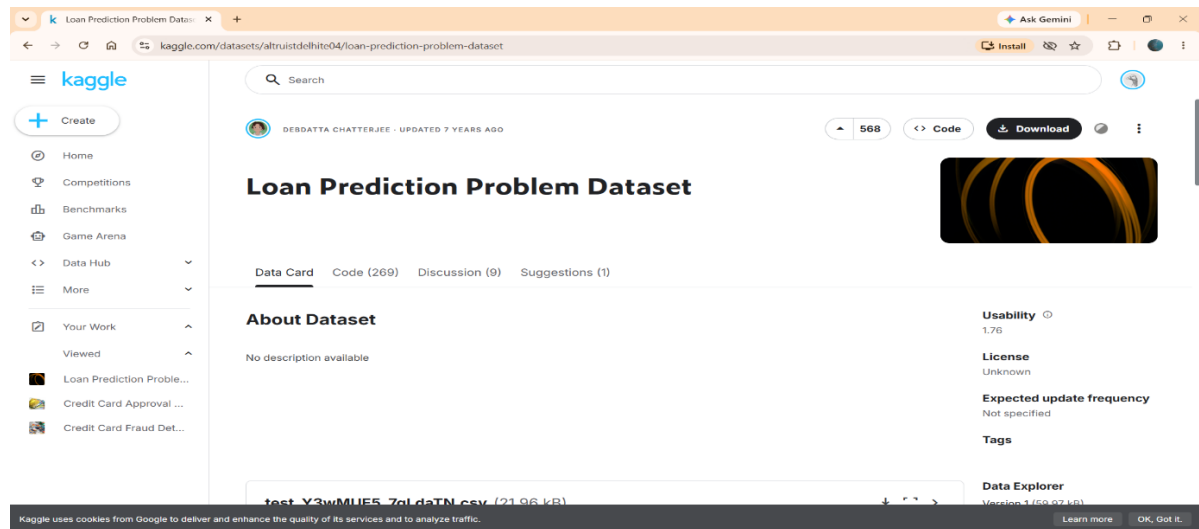
Before training the machine learning model, a suitable loan prediction dataset must be collected and analyzed. Follow the steps below to obtain and understand the dataset used in the Smart Lender project.

Step 1 — Download the Dataset

Visit the Kaggle website and download the Loan Prediction Dataset.

Website:

<https://www.kaggle.com/>

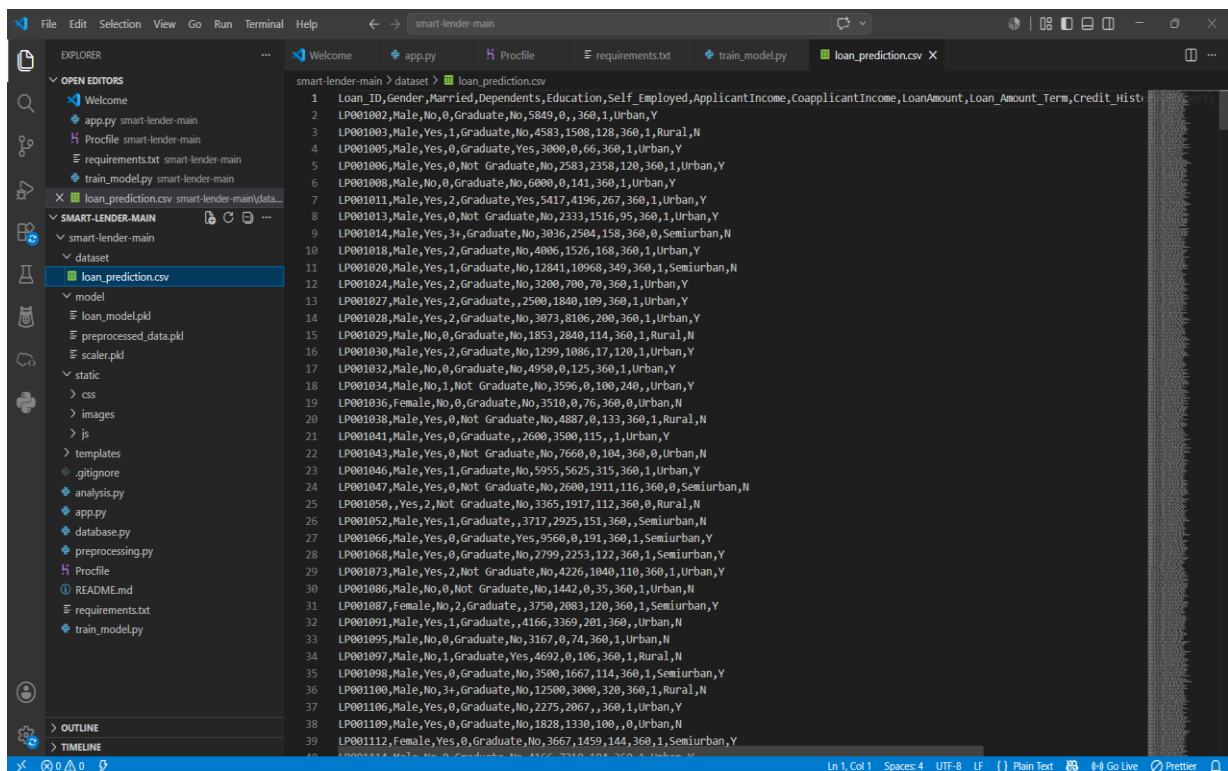


Step 2 — Create the Project Folder

Create a project directory named:

Smart Lender

Inside the project folder, create a folder named **dataset** and place the downloaded CSV file inside it.



Step 3 — Load the Dataset

Open the project in Visual Studio Code and use Pandas to load the dataset.

Example:

```
import pandas as pd
```

```
df = pd.read_csv("dataset/loan_prediction.csv")
```

Step 4 — Analyze the Dataset

Perform initial dataset exploration by checking

- Number of records
- Number of attributes
- Missing values
- Data types
- Target variable distribution

This helps identify preprocessing requirements before training the machine learning model.

Step 5 — Verify Dataset Successfully Loaded

Display the first few records using

```
df.head()
```

Verify that all columns are loaded correctly before proceeding to preprocessing.

Activity 1.2: Research and Select the Appropriate Generative AI Model

Here are the things that needed to research to select a best model for the project:

- **Understand the Project Requirements:** Review the specific needs of the CaptionAI application, focusing on the types of content it will generate (captions, hashtags, and calls-to-action across Instagram and LinkedIn).
- **Explore Groq's Model Documentation:** Visit the Groq documentation to examine the available LLM models, including their functionalities, context windows, speed, and limitations.
- **Evaluate Model Performance:** Compare models based on response quality, generation speed, and relevance to social media copywriting tasks.
- **Select the Optimal Model:** Choose **llama-3.3-70b-versatile** for its balance of creative output quality and generation speed, set with a temperature of 0.8 and max_tokens of 2048 for rich, varied captions.

Activity 1.3: Define the Architecture of the Application

- **Draft an Architectural Diagram:** Create a visual representation of the application architecture, including components like the frontend (HTML, CSS, JavaScript), Flask backend, and Groq API integration.
- **Detail Frontend Functionality:** Outline how users interact with the application entering product/campaign information, selecting a platform (Instagram or LinkedIn), and choosing a tone (Professional, Casual, or Promotional).

- **Outline Backend Responsibilities:** Specify how the Flask backend handles incoming POST requests to /generate, validates and sanitizes user input, builds a structured prompt, and communicates with the Groq API.
- **Describe AI Integration Points:** Define how the backend constructs prompts using TONE_DESCRIPTIONS and PLATFORM_TIPS dictionaries, calls the Groq API, parses the JSON response via extract_json(), and returns structured results to the frontend.

Activity 1.4: Set Up the Development Environment

- **Install Python and Pip:** Ensure Python 3.8+ is installed on your machine along with pip for dependency management.
- **Create a Virtual Environment:** Set up a virtual environment using venv to isolate project dependencies:

```
D:\projects>python -m venv myenv  
D:\projects>"d:\projects\venv\Scripts\activate"  
(myenv) D:\projects> pip install -r requirements.txt
```

- **Install Flask:** Use pip to install Flask:

```
(myenv) D:\projects> pip install Flask==3.0.3
```

- **Install Groq SDK:** Install the Groq Python library to interact with the Groq API:

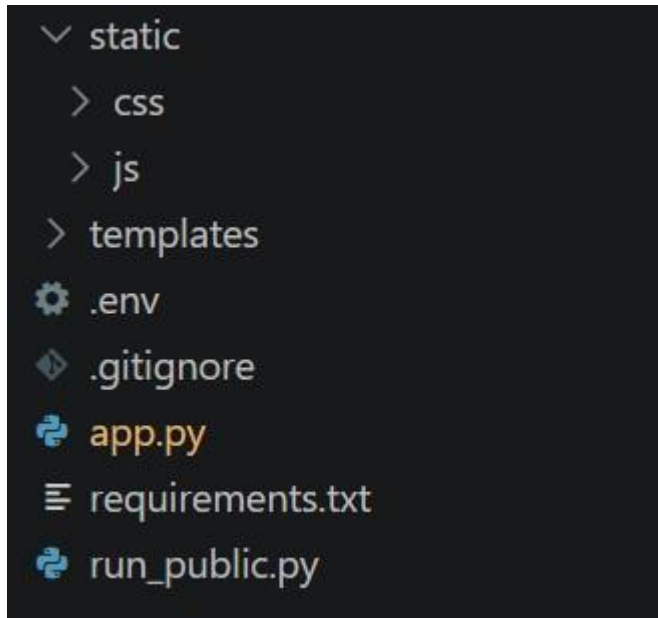
```
(myenv) D:\projects> pip install groq==0.11.0
```

- **Install python-dotenv:** Install dotenv support for secure API key management:

```
(myenv) D:\projects> pip install python-dotenv==1.0.1
```

- **Configure the API Key:** Create a .env file in the project root and add your Groq API key:
GROQ_API_KEY=your_groq_api_key_here

- **Set Up Application Structure:** Create the project directory structure including folders for templates, static files (CSS, JS), and main application files (app.py, requirements.txt, run_public.py).



Milestone 2: Core Functionalities Development

Activity 2.1: Develop Core Content Generation Features

Implement the three core generation features Captions, Hashtags, and CTAs as a unified AI response driven by a single structured prompt.

- **Caption Generator:** Generates three unique, platform-appropriate captions. For Instagram/Casual tone, includes relevant emojis and conversational language. For LinkedIn/Professional tone, produces insight-led, value-driven copy.
- **Hashtag Suggester:** Produces ten hashtags per request a curated mix of high-reach popular tags and targeted niche tags relevant to the product or campaign.
- **CTA Generator:** Returns three short (under 10 words), punchy, action-oriented call-to-action phrases calibrated to the selected platform and tone.

Activity 2.2: Implement the Flask Backend

Define Routes in Flask:

- Set up routes in app.py for the home page and content generation endpoint.

Process User Input:

- Create a form in index.html for users to submit their product/campaign information, platform, and tone.
- Use Flask's request.get_json() to retrieve JSON data submitted from the frontend via AJAX.
- Validate that product_info is non-empty and within the 2000-character limit before processing.

Integrate API Calls:

- Within the /generate route handler, call the build_prompt() function to construct a structured prompt combining tone descriptions and platform-specific tips.
- Submit the prompt to the Groq API using client.chat.completions.create() with the llama-3.3-70b-versatile model.
- Parse and validate the JSON response using extract_json(), which handles direct JSON, markdown code blocks, and raw JSON object extraction as fallback strategies.

Milestone 3: App.py Development

Milestone 3 focuses on creating and refining the core logic of the app.py file, which serves as the backbone of the CaptionAI application. In this milestone, you'll set up each feature's functionality through Flask routes, establish reliable prompt construction and API interaction, and conduct unit testing to ensure each feature performs as expected.

Activity 3.1: Writing the Main Application Logic in app.py

Define the Core Routes in app.py:

- Establish routes for CaptionAI's two endpoints, each serving a distinct purpose: ◦ / — Home page route that renders the main index.html interface

```
@app.route("/")
def index():
    return render_template("index.html")
```

- **/generate** — POST endpoint that accepts JSON input, invokes the AI, and returns structured content

```
@app.route("/generate", methods=["POST"])
def generate():
    data = request.get_json()
    product_info = data.get("product_info", "").strip()
    platform = data.get("platform", "instagram").lower()
```

```
tone = data.get("tone", "professional").lower()

if not product_info:
    return jsonify({"error": "Product information is required."}), 400
if len(product_info) > 2000:
    return jsonify({"error": "Input too long. Please keep it under 2000 characters."}), 400
try:
    prompt = build_prompt(product_info, platform, tone)

    chat_completion = client.chat.completions.create(
        messages=[
            {
                "role": "system",
                "content": "You are a social media content expert. Always respond with valid JSON only, no markdown formatting."
            },
            {
                "role": "user",
                "content": prompt
            }
        ],
        model="llama-3.3-70b-versatile",
        temperature=0.8,
        max_tokens=2048,
    )

    response_text = chat_completion.choices[0].message.content
    result = extract_json(response_text)
    # Validate structure
    if not all(k in result for k in ["captions", "hashtags", "ctas"]):
        raise ValueError("Invalid response structure from AI")
    return jsonify({
        "success": True,
        "captions": result["captions"],
        "hashtags": result["hashtags"],
        "ctas": result["ctas"],
        "platform": platform,
        "tone": tone
    })
except ValueError as e:
    import traceback
    traceback.print_exc()
    return jsonify({"error": f"Failed to parse AI response: {str(e)}"}), 500
except Exception as e:
```

```
import traceback

traceback.print_exc()

return jsonify({"error": f"Generation failed: {str(e)}"}), 500
```

Set Up Route Handlers for Each Feature:

- For the /generate route, implement logic that reads product_info, platform, and tone from the incoming JSON body using request.get_json().
- Apply input validation: return a 400 error if product_info is empty or exceeds 2000 characters.
- Route AJAX requests from the frontend to the /generate endpoint, which processes data and returns a JSON response.

Define Prompt Engineering Helpers:

- **TONE_DESCRIPTIONS** dictionary: maps professional, casual, and promotional tones to detailed copywriting instructions embedded in the prompt.

```
TONE_DESCRIPTIONS = {

    "professional": "formal, authoritative, and business-oriented. Use clear, concise language that conveys expertise and credibility.",

    "casual": "friendly, conversational, and relatable. Use informal language, emojis, and a warm tone that feels personal.",

    "promotional": "exciting, persuasive, and action-driven. Use power words, urgency, and compelling offers to drive engagement."

}
```

- **PLATFORM_TIPS** dictionary: maps instagram and linkedin to platform-specific content guidelines (character limits, style expectations).

```
PLATFORM_TIPS = {
```

```
"instagram": "optimized for Instagram – visually descriptive, emotionally engaging, with strong
visual storytelling. Max 2200 characters.",

"linkedin": "optimized for LinkedIn – professional insights, value-driven, thought leadership
style. Max 3000 characters."
}
```

- **build_prompt()**: assembles a structured prompt string from the product info, selected platform tip, and tone description, instructing the model to return strictly valid JSON.

```
def build_prompt(product_info: str, platform: str, tone: str) -> str:

    tone_desc = TONE_DESCRIPTIONS.get(tone, "engaging")

    platform_tip = PLATFORM_TIPS.get(platform, "social media")

    return f"""You are an expert social media content strategist and copywriter.
```

Integrate the Groq AI Responses in Each Function:

- Call `client.chat.completions.create()` with a system message enforcing JSON-only responses and a user message containing the assembled prompt.
- **extract_json()**: a robust parser that first attempts `json.loads()`, then strips markdown code fences (````json` blocks), and finally uses a regex to locate a raw JSON object ensuring reliable parsing across model response variations.

```
def extract_json(text: str) -> dict:

    """Extract JSON from LLM response, handling markdown code blocks."""

    # Try direct parse first

    try:

        return json.loads(text)

    except json.JSONDecodeError:

        pass

    # Try extracting from markdown code block

    match = re.search(r'```(?:json)?\s*([\s\S]*)```', text)

    if match:

        try:
```

```
        return json.loads(match.group(1).strip())
    except json.JSONDecodeError:
        pass
    match = re.search(r'\{[^\s\S]*\}', text)
    if match:
        try:
            return json.loads(match.group(0))
        except json.JSONDecodeError:
            pass
    raise ValueError("Could not extract valid JSON from response")
```

- Validate that the parsed result contains all three required keys — captions, hashtags, and ctas — before returning it to the frontend.
- **Home route:** / → renders index.html/generate (POST) → accepts JSON, returns {success,captions, hashtags, ctas, platform, tone}

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for CaptionAI. This involves designing a glassmorphism-style layout with responsive HTML and CSS to enhance usability, and creating dynamic content rendering with vanilla JavaScript to display AI-generated captions, hashtags, and CTAs based on user interactions.

Activity 4.1: Designing and Developing the User Interface

Set Up the Base HTML Structure:

- Develop the main HTML file (index.html) as the single-page interface, including a header with the CaptionAI brand logo and tagline, an input section, and a results section.
- Use semantic HTML elements to keep the structure organized and ensure the interface is accessible for all users.
- Integrate Font Awesome icons for platform indicators and result section headings, and Google Fonts (Inter) for clean typography.

Design a Responsive Layout Using CSS:

- Create a CSS stylesheet (static/css/style.css) implementing a dark glassmorphism aesthetic with animated background blobs (blob-1, blob-2, blob-3) for visual depth.
- Use flexbox and grid layouts to organize the results into a two-column structure (captions on the left, hashtags and CTAs on the right).
- Add media queries to ensure the layout adapts across desktop, tablet, and mobile screen sizes.

Create Separate UI Components for Each Output Type:

- **Caption Cards:** individual cards per caption, each with a one-click copy-to-clipboard button that provides visual feedback (checkmark + green highlight for 2 seconds).
- **Hashtag Chips:** rendered as pill-shaped chip elements inside a glass card, with automatic # prefix validation.
- **CTA List:** displayed as a clean unordered list inside a separate glass card with a bullhorn icon header.

Activity 4.2: Creating Dynamic Content Rendering with JavaScript

Handle Form Submission Asynchronously:

- Attach a submit event listener to the generator form that prevents default form submission and sends the data as a JSON POST request to /generate via the Fetch API.
- During loading, disable the submit button and show an animated CSS loader in place of the button text and arrow icon.

Render Results Dynamically:

- Implement the renderResults(data) function in main.js, which dynamically creates and injects DOM elements for captions, hashtags, and CTAs into their respective containers.
- After rendering, automatically smooth-scroll the viewport to the results section using scrollToView().

Restore UI State After Generation:

- In the finally block of the async handler, re-enable the submit button and restore the original button text and icon regardless of success or failure.

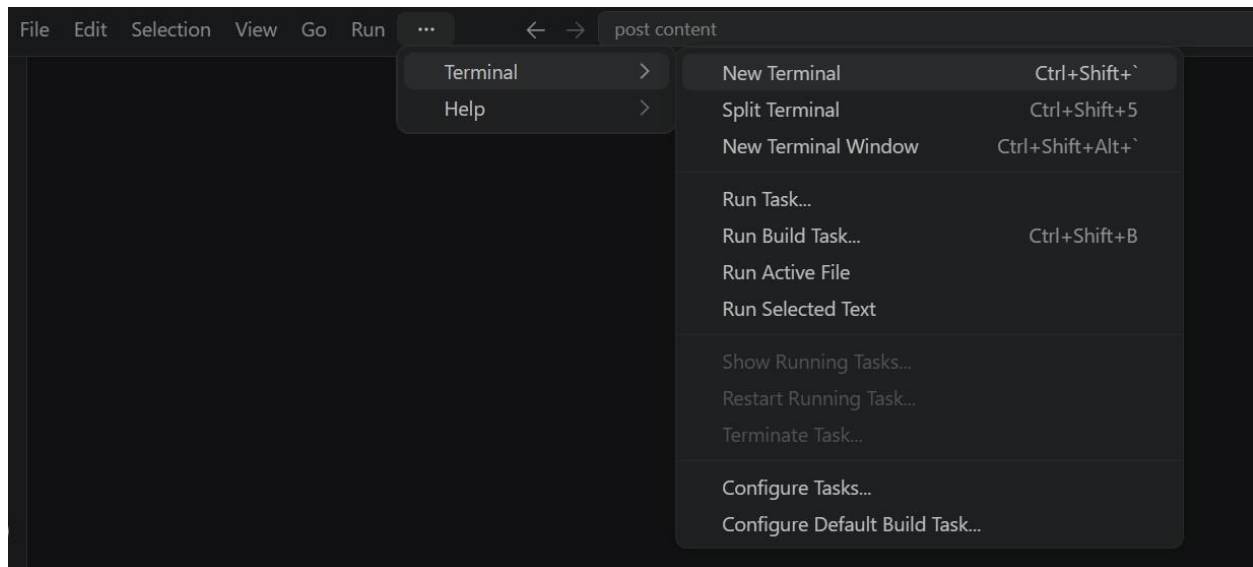
Milestone 5: Deployment

In Milestone 5, the focus is on deploying the CaptionAI application first locally to verify correct operation, and then publicly via Ngrok to share the app over a secure, accessible URL. This involves configuring the server environment, managing dependencies, and launching the app for real-world use.

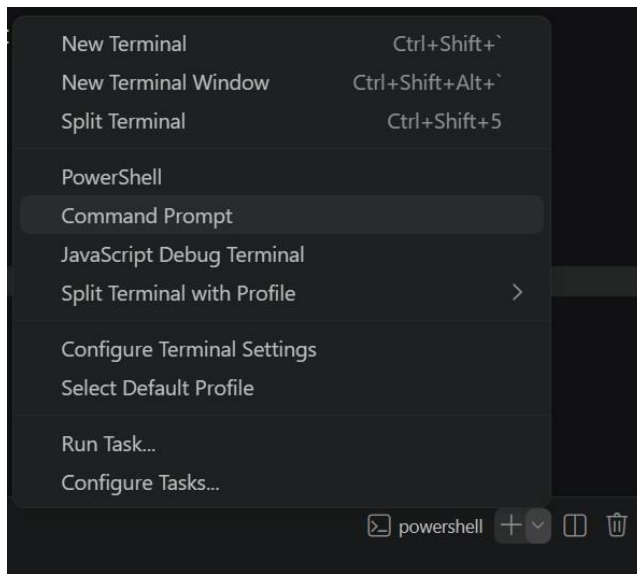
Activity 5.1: Preparing the Application for Local Deployment

Set Up a Virtual Environment:

- Open a New Terminal



- Then open “Command Prompt”



- Create and activate a virtual environment, then install all required dependencies from requirements.

```
D:\projects>python -m venv myenv
D:\projects>"d:\projects\venv\Scripts\activate"
(myenv) D:\projects>pip install -r requirements.txt
```

Configure Environment Variables:

- Create a .env file in the project root and add your Groq API key. The app loads this automatically using python-dotenv at startup: GROQ_API_KEY=your_groq_api_key_here

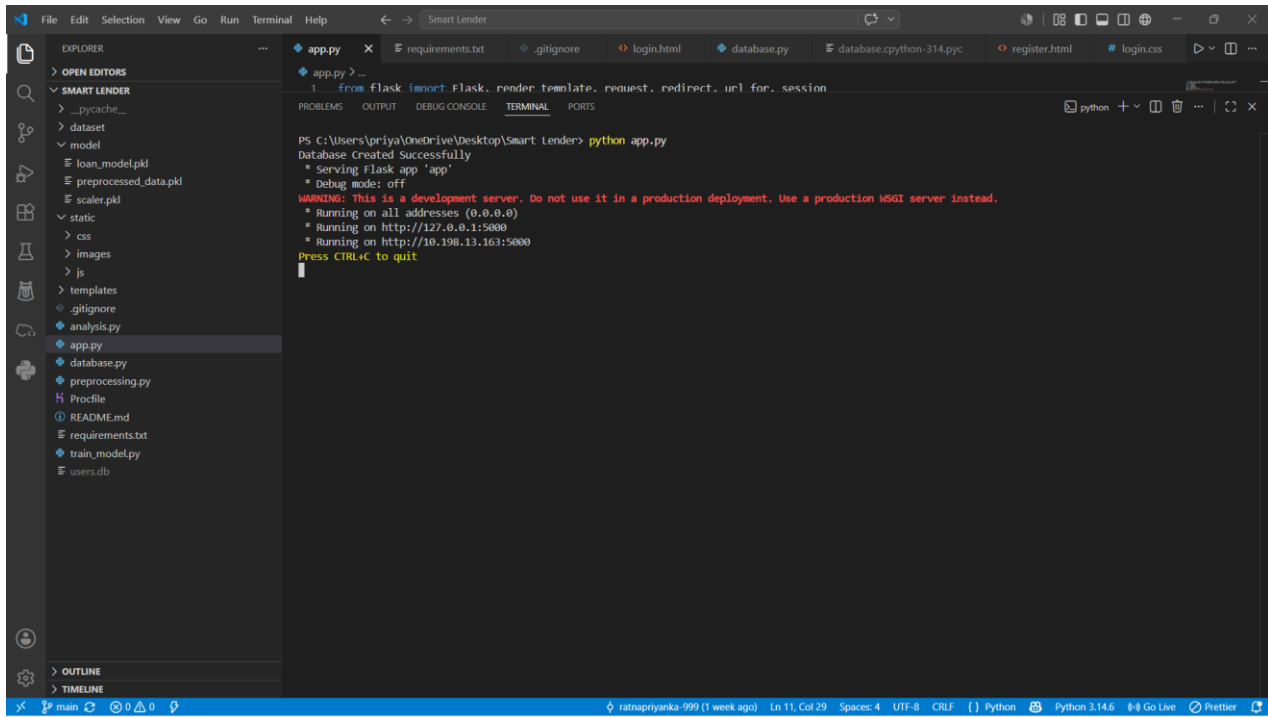
Activity 5.2: Local Testing and Verification

Start the Flask Development Server:

- Run the application locally using:

```
(myenv) D:\projects>python app.py
```

- Once running, open a browser and navigate to “http://127.0.0.1:5050” to access the app.
- Interact with the caption generator test all three tones (Professional, Casual, Promotional) and both platforms (Instagram, LinkedIn) to verify the AI responses are correctly generated, parsed, and rendered.



```

PS C:\Users\priya\OneDrive\Desktop\Smart Lender> python app.py
Database Created Successfully
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.198.13.163:5000
Press CTRL+C to quit
  
```

Activity 5.3: Public Deployment via Ngrok

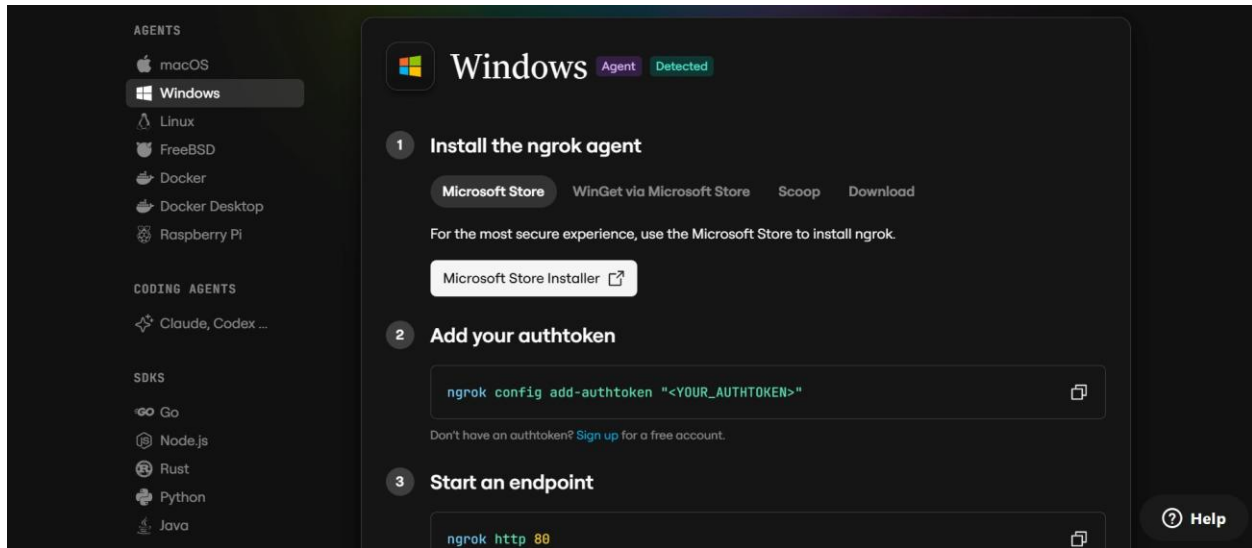
Ngrok creates a secure tunnel from a public URL to your local Flask server, making the app accessible from anywhere without a cloud hosting provider.

Install Ngrok and pyngrok:

- Install the pyngrok Python wrapper, which manages Ngrok programmatically within your Python project:

```
D:\projects>pip install pyngrok
```

- Download and install the Ngrok client from <https://ngrok.com/download>.
- Click on “Microsoft Store Installer”.



Configure Your Ngrok Authtoken:

- Sign up at ngrok.com and copy your authtoken from the Ngrok dashboard.
- The `run_public.py` script sets the authtoken automatically using:

```
D:\projects>ngrok.set_auth_token("YOUR_NGROK_AUTHTOKEN")
```

- Replace the `TOKEN` value in `run_public.py` with your own Ngrok authtoken before running.

Run the Public Deployment Script:

- Instead of running `app.py` directly, launch `run_public.py` to start both the Ngrok tunnel and the Flask server:

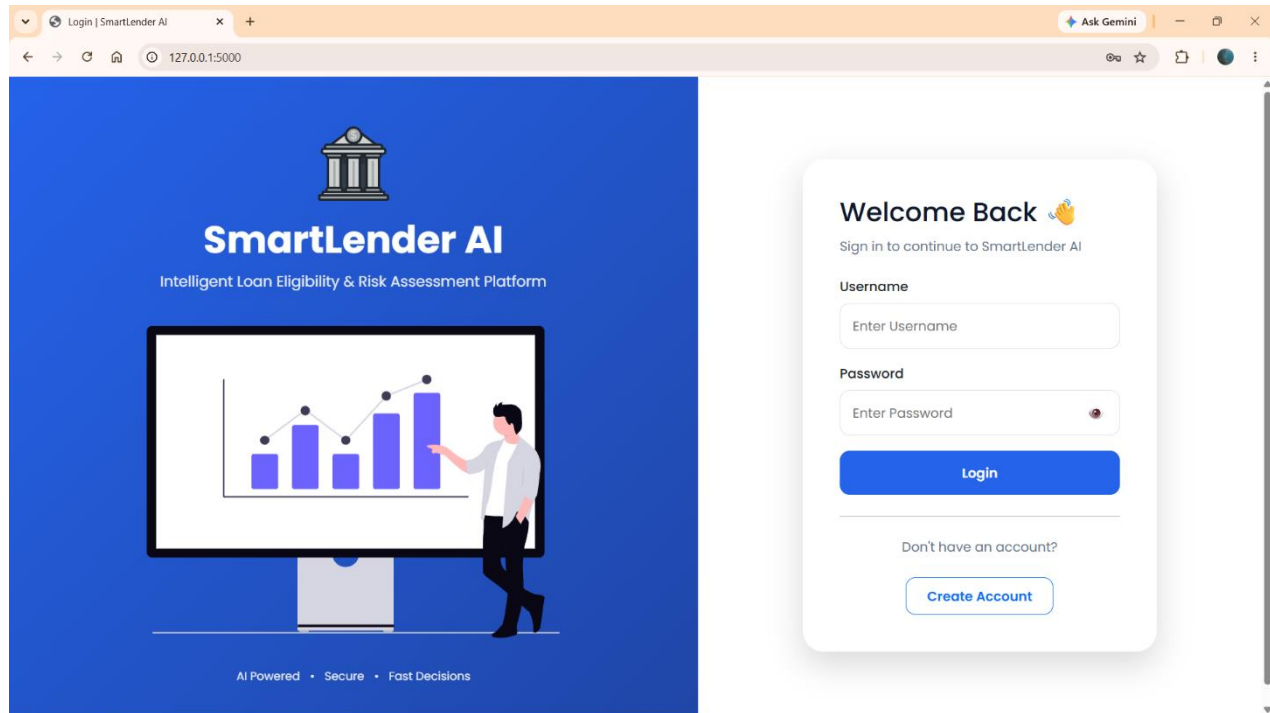
```
D:\projects>python run_public.py
```

- The script opens an HTTP tunnel on port 5000 and prints the live public URL in the terminal:

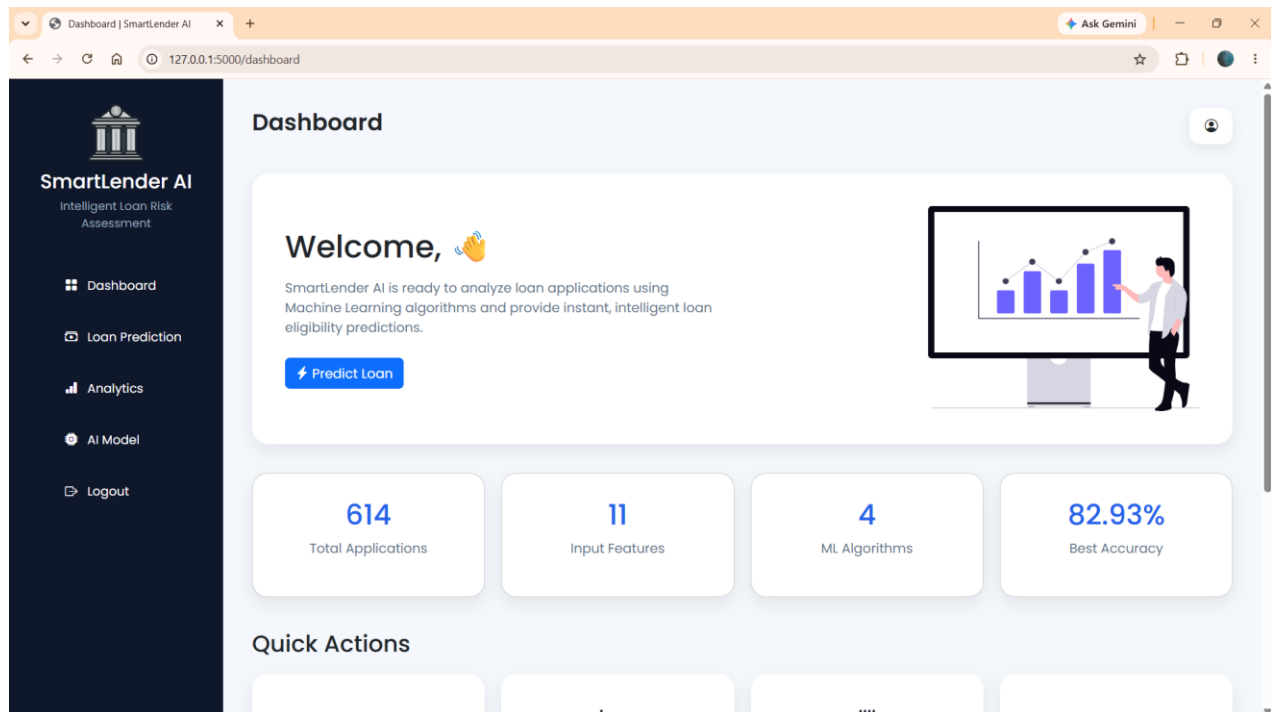
```
===== YOUR
PUBLIC WEBSITE URL IS LIVE!
URL: https://xxxx-xx-xx-xxx-xx.ngrok-free.app
=====
```
- Share this URL with anyone who can access your CaptionAI app directly from their browser without any installation.

Important Notes:

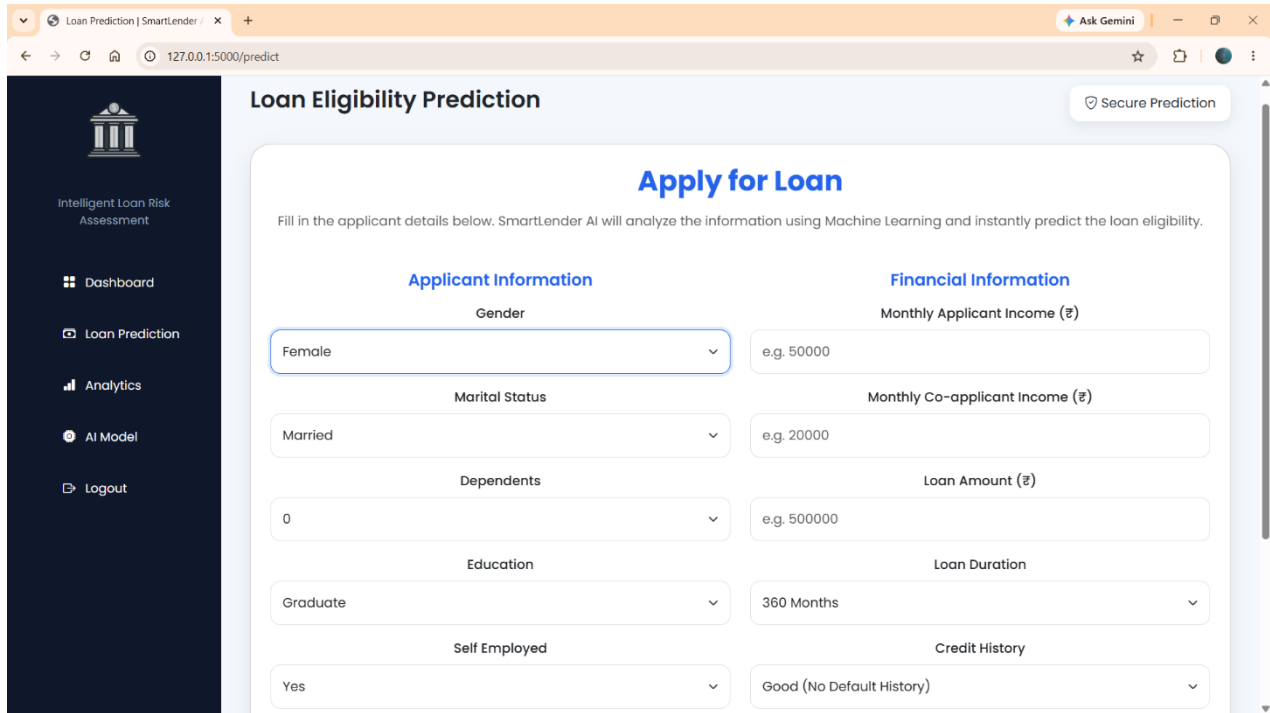
- **debug=False** is required in `run_public.py` to prevent Flask from spawning a reloader process, which would interfere with Ngrok's tunnel management.
- The public URL changes each time you restart `run_public.py` (on the free Ngrok plan). For a persistent URL, upgrade to a paid Ngrok plan and configure a reserved domain.
- Ngrok free tier sessions expire after a few hours of inactivity. Restart `run_public.py` to get a new URL when this happens.



Description: The single-page interface of Smart Lender serves as both the landing page and the generation tool. It features a dark glassmorphism design with animated background blobs, a header displaying the Smart Lender brand (wand-magic-sparkles icon + logo), and a tagline. The main section contains the input form, platform toggle, tone selector, and the AI-generated results panel all on one seamless page.



Input Section:

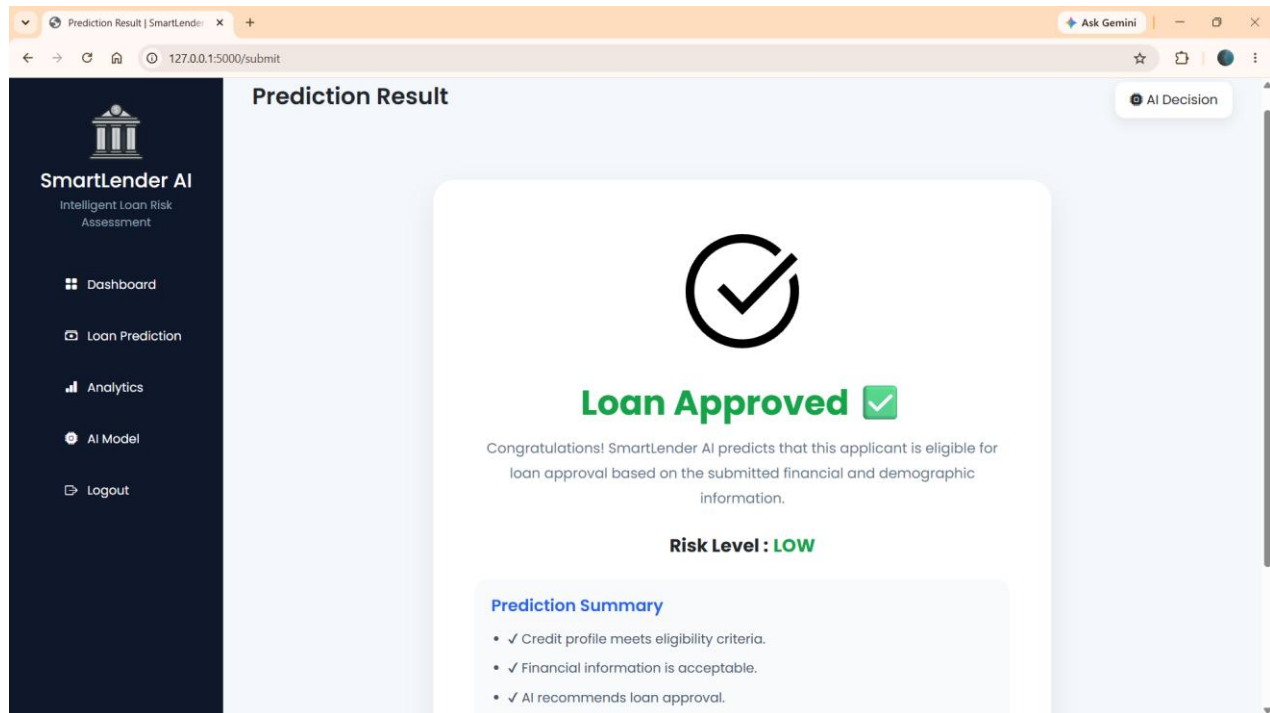


The screenshot shows a web browser window with the URL `127.0.0.1:5000/predict`. The page title is "Loan Eligibility Prediction". On the left is a dark blue sidebar with a logo and navigation links: "Intelligent Loan Risk Assessment", "Dashboard", "Loan Prediction", "Analytics", "AI Model", and "Logout". The main content area has a light blue header with the title "Loan Eligibility Prediction" and a "Secure Prediction" button. Below this is a white card titled "Apply for Loan" with the instruction: "Fill in the applicant details below. SmartLender AI will analyze the information using Machine Learning and instantly predict the loan eligibility." The form is divided into two columns: "Applicant Information" and "Financial Information".

Applicant Information		Financial Information	
Gender	Female	Monthly Applicant Income (₹)	e.g. 50000
Marital Status	Married	Monthly Co-applicant Income (₹)	e.g. 20000
Dependents	0	Loan Amount (₹)	e.g. 500000
Education	Graduate	Loan Duration	360 Months
Self Employed	Yes	Credit History	Good (No Default History)

Description: A glass-styled card containing the main generation form. Users enter their product, service, or campaign description in a textarea (up to 2000 characters), select their target platform via a toggle button group (Instagram or LinkedIn), choose their tone of voice (Professional, Casual, or Promotional), and click the Generate Content button. While the AI processes the request, the button transitions to an animated loading spinner.

Results Section:



Description: Revealed dynamically after generation, this section displays the AI output in a two-column grid. The left column shows three caption cards, each with the generated caption text and a copy-to-clipboard button. The right column contains two glass cards: one with ten hashtag chips (prefixed with #) and one with three CTA phrases in a list format. The viewport automatically smooth-scrolls to this section upon content generation.

Conclusion:

Samrt Lender is a generative AI platform built with Flask and styled with a glassmorphism UI that streamlines social media content creation. It uses Groq's high-speed inference API with the LLaMA 3.3-70B Versatile model to instantly generate tailored captions, hashtags, and calls-to-action for Instagram and LinkedIn across professional, casual, and promotional tones. The project, which included model selection, development, and deployment via Ngrok, highlights how targeted AI and a structured framework can boost productivity for marketers and creators, with potential for future scalability. Looking ahead, CaptionAI offers significant opportunities for expansion, such as integrating multi-language support, adding image analysis capabilities for visual-context generation, and implementing user authentication to save generation history. By continuously refining the underlying model prompts and enhancing the frontend accessibility, the platform is well-positioned to evolve from a specialized tool into a comprehensive social media management assistant for teams of all sizes.